

Python Practical Workbook: 50 Exercises with Solutions (Functions, Lists, Sets, Tuples, Dictionaries)

This workbook provides a comprehensive set of fifty practical exercises designed to enhance proficiency in core Python programming concepts. These questions focus on functional programming, data structure manipulation, and the unique properties of lists, sets, tuples, and dictionaries. By working through these exercises, developers can solidify their understanding of Python's versatile toolset for data handling and logic implementation.

Each exercise includes a problem statement, a complete code solution, and a detailed explanation of the concepts involved, including common pitfalls to avoid.

I. Python Functions

1. Rectangle Area Calculator *Problem:* Develop a function `calculate_area(length, width)` that computes the area of a rectangle. Ensure the function includes a descriptive docstring. *Solution:*

```
def calculate_area(length, width):  
    """  
    Calculates the area of a rectangle.  
  
    Args:  
        length (float): The length of the rectangle.  
        width (float): The width of the rectangle.  
  
    Returns:  
        float: The calculated area (length * width).  
    """  
    return length * width  
  
print(calculate_area(5, 10)) # Output: 50
```

Explanation: The `""" """` block immediately following the function definition is a docstring. It's crucial for documenting what the function does, its arguments, and its return value. *Pitfall:* Forgetting to `return` the value; a function without a `return` statement implicitly returns `None`.

2. Customizable Greetings Problem: Create a function `greet(name, greeting="Hello")` that outputs a personalized message. Demonstrate how the default argument behaves when omitted versus when provided. *Solution:*

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")           # Output: Hello, Alice!
greet("Bob", greeting="Hi") # Output: Hi, Bob!
```

Explanation: `greeting="Hello"` sets a default value. If the caller doesn't provide this argument, "Hello" is used. *Pitfall:* Default arguments are evaluated only once when the function is defined. Never use mutable objects (like `[]` or `{}`) as default arguments, as they will be shared across all calls.

3. Dynamic Summation Problem: Implement a function `sum_all(*args)` that accepts any number of numerical inputs and returns their cumulative sum. *Solution:*

```
def sum_all(*args):
    total = 0
    for num in args:
        total += num
    return total
# Alternatively, just use the built-in: return sum(args)

print(sum_all(1, 2, 3, 4)) # Output: 10
```

Explanation: The `*args` syntax packs any number of positional arguments into a tuple named `args` inside the function.

4. User Profile Generator Problem: Design a function `create_profile(name, age, **kwargs)` that takes mandatory identity details and optional attributes like city or profession, printing a formatted summary. *Solution:*

```
def create_profile(name, age, **kwargs):
    profile = {"name": name, "age": age}
    profile.update(kwargs)
    for key, value in profile.items():
        print(f"{key.capitalize()}: {value}")

create_profile("Charlie", 30, city="New York", profession="Engineer")
```

Explanation: `**kwargs` collects any extra keyword arguments passed to the function into a dictionary named `kwargs`.

5. Recursive Factorials *Problem:* Write a recursive implementation of a function `factorial(n)` to calculate the factorial of a given non-negative integer. *Solution:*

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5)) # Output: 120
```

Explanation: Recursion occurs when a function calls itself. *Pitfall:* Always ensure there is a “base case” (here, `n == 0` or `n == 1`) to stop the recursion; otherwise, you will hit a `RecursionError` (stack overflow).

6. Lambda-Based Sorting *Problem:* Given a list of dictionaries representing students with ‘name’ and ‘age’ keys, utilize a lambda function within the `sort()` method to organize the list by age. *Solution:*

```
students = [{'name': 'Alice', 'age': 25}, {'name': 'Bob', 'age': 20}, {'name': 'Charlie', 'age': 30}]
students.sort(key=lambda student: student['age'])
print(students)
```

Explanation: `lambda student: student['age']` is an anonymous, inline function. The `sort()` method uses the value returned by this lambda (the age) to determine the sorting.

order.

7. Functional Filtering *Problem:* Use the `filter()` function with a lambda expression to get all numbers greater than 5 from a list `[1, 7, 3, 9, 2, 8]`. *Solution:*

```
numbers = [1, 7, 3, 9, 2, 8]
filtered_numbers = list(filter(lambda x: x > 5, numbers))
print(filtered_numbers) # Output: [7, 9, 8]
```

Explanation: `filter()` constructs an iterator from elements of an iterable for which a function returns true. We convert it to a list to see the results.

8. Function Decorator with Arguments *Problem:* Create a decorator `repeat(num_times)` that makes a function run `num_times` times. *Solution:*

```
def repeat(num_times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(num_times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(num_times=3)
def say_hello():
    print("Hello!")

say_hello() # Output: Hello! (3 times)
```

Explanation: This is a decorator factory. `repeat` takes an argument and returns the actual decorator, which then wraps the function. This allows for configurable decorators.

9. Generator for Even Numbers *Problem:* Write a generator function `even_numbers(limit)` that yields even numbers up to a specified limit. *Solution:*

```
def even_numbers(limit):
    for i in range(0, limit + 1, 2):
        yield i

print(list(even_numbers(10))) # Output: [0, 2, 4, 6, 8, 10]
```

Explanation: Generators are memory-efficient as they produce items one by one and only calculate the next item when requested, rather than building a full list in memory.

10. Function to Validate Email Format *Problem:* Write a function `is_valid_email(email)` that checks if a string is a basic email format (contains '@' and a '.' after '@'). *Solution:*

```
def is_valid_email(email):
    if '@' in email:
        parts = email.split('@')
        if len(parts) == 2 and '.' in parts[1]:
            return True
    return False

print(is_valid_email("test@example.com")) # Output: True
print(is_valid_email("invalid-email"))    # Output: False
```

Explanation: This function performs a basic check for the presence of '@' and a '.' in the domain part. For robust email validation, regular expressions are typically used.

II. Working with Lists

11. List Initialization and Access *Problem:* Create a list `fruits = ["apple", "banana", "cherry"]`. Access and print the second element. Add "orange" to the end of the list. *Solution:*

```
fruits = ["apple", "banana", "cherry"]
print(fruits[1]) # Output: banana (0-indexed)
fruits.append("orange")
print(fruits)    # Output: ['apple', 'banana', 'cherry', 'orange']
```

Explanation: Lists are zero-indexed. `append()` adds an element to the very end of the list in-place.

12. List Slicing Problem: Given `numbers = [10, 20, 30, 40, 50, 60]`, extract a sub-list containing `[30, 40, 50]` using slicing. *Solution:*

```
numbers = [10, 20, 30, 40, 50, 60]
sub_list = numbers[2:5]
print(sub_list) # Output: [30, 40, 50]
```

Explanation: Slicing syntax is `list[start:stop]`. It includes the `start` index (2, which is the 3rd element) but excludes the `stop` index (5, which is the 6th element).

13. List Comprehension (Even Numbers) Problem: Use list comprehension to create a new list containing only the even numbers from `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`. *Solution:*

```
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens) # Output: [2, 4, 6, 8, 10]
```

Explanation: List comprehensions provide a concise way to create lists. The syntax is `[expression for item in iterable if condition]`.

14. List Manipulation (Remove Duplicates) Problem: Write a function `remove_duplicates(input_list)` that takes a list with potential duplicates and returns a new list with only unique elements, preserving order. *Solution:*

```
def remove_duplicates(input_list):
    seen = set()
    result = []
    for item in input_list:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_duplicates([1, 2, 2, 3, 1, 4])) # Output: [1, 2, 3, 4]
```

Explanation: Using an auxiliary set (`seen`) to track encountered elements allows us to build a new list with unique elements while preserving their original order.

15. Nested Lists (Matrix Access) Problem: Create a nested list representing a 3x3 matrix. Access and print the element in the second row, third column. *Solution:*

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
element = matrix[1][2] # Row index 1 (second row), Column index 2 (third column)  
print(element) # Output: 6
```

Explanation: Nested lists are accessed by chaining index brackets. `matrix[1]` gets the second inner list, and `[2]` then accesses its third element.

16. List Sorting (Multiple Criteria) Problem: Sort a list of strings `words = ["banana", "apple", "cherry", "date"]` alphabetically. Then, sort it by the length of the words. *Solution:*

```
words = ["banana", "apple", "cherry", "date", "fig"]  
words.sort() # Sorts alphabetically first  
words.sort(key=len) # Then sorts by length (stable sort preserves alphabetical order)  
print(words) # Output: ['fig', 'date', 'apple', 'banana', 'cherry']
```

Explanation: Python's `sort()` is stable, meaning that if two elements have the same sort key, their relative order is preserved. Sorting by length after alphabetical sort ensures that words of the same length remain alphabetically ordered.

17. List Concatenation and Repetition Problem: Concatenate two lists `list1 = [1, 2]` and `list2 = [3, 4]`. Then, create a new list by repeating `list1` three times. *Solution:*

```
list1 = [1, 2]
list2 = [3, 4]
combined = list1 + list2
repeated = list1 * 3
print(combined) # Output: [1, 2, 3, 4]
print(repeated) # Output: [1, 2, 1, 2, 1, 2]
```

Explanation: The `+` operator creates a new list by concatenating two lists. The `*` operator creates a new list by repeating the elements of the original list a specified number of times.

18. Checking Element Existence Problem: Write a function `check_item(item, my_list)` that returns `True` if `item` is in `my_list`, otherwise `False`. *Solution:*

```
def check_item(item, my_list):
    return item in my_list

print(check_item(3, [1, 2, 3, 4])) # Output: True
print(check_item(5, [1, 2, 3, 4])) # Output: False
```

Explanation: The `in` operator is the most Pythonic and efficient way to check for membership in a list.

19. List Reversal Problem: Reverse a list `data = [1, 2, 3, 4, 5]` in-place and also create a new reversed list without modifying the original. *Solution:*

```
data = [1, 2, 3, 4, 5]
# In-place reversal
data.reverse()
print(f"In-place reversed: {data}") # Output: [5, 4, 3, 2, 1]

# Create a new reversed list
original_data = [1, 2, 3, 4, 5]
new_reversed_data = original_data[::-1]
print(f"New reversed list: {new_reversed_data}") # Output: [5, 4, 3, 2, 1]
print(f"Original list unchanged: {original_data}") # Output: [1, 2, 3, 4, 5]
```


Explanation: `list.reverse()` modifies the list directly and returns `None`. Slicing with `[::-1]` creates a new reversed list, leaving the original list untouched.

20. List as a Stack/Queue Problem: Demonstrate how a Python list can be used as a stack (LIFO) and a queue (FIFO) using `append()`, `pop()`, and `insert()` / `pop(0)`. *Solution:*

```
# Stack (LIFO - Last In, First Out)
stack = []
stack.append(1) # Push
stack.append(2)
print(f"Stack after pushes: {stack}") # Output: [1, 2]
print(f"Popped from stack: {stack.pop()}") # Output: 2
print(f"Stack after pop: {stack}") # Output: [1]

# Queue (FIFO - First In, First Out) - Note: collections.deque is better
queue = []
queue.append(1) # Enqueue
queue.append(2)
print(f"Queue after enqueues: {queue}") # Output: [1, 2]
print(f"Dequeued from queue: {queue.pop(0)}") # Output: 1
print(f"Queue after dequeue: {queue}") # Output: [2]
```

Explanation: For a stack, `append()` adds to the end and `pop()` removes from the end. For a queue, `append()` adds to the end, but `pop(0)` removes from the beginning. *Pitfall:* `pop(0)` on a list is inefficient for large lists ($O(n)$) because all subsequent elements must be shifted. For true queue performance, `collections.deque` is preferred.

III. Mastery of Sets

21. Set Creation and Basic Operations Problem: Create a set `colors = {"red", "blue", "green"}`. Add "yellow" to the set. Try adding "red" again and observe the result. Remove "blue" from the set. *Solution:*

```

colors = {"red", "blue", "green"}
print(f"Initial set: {colors}")
colors.add("yellow")
print(f"After adding yellow: {colors}")
colors.add("red") # Adding a duplicate has no effect
print(f"After adding red (duplicate): {colors}")
colors.remove("blue")
print(f"After removing blue: {colors}")

```

Explanation: Sets only store unique elements. Adding an existing element has no effect.

`remove()` raises a `KeyError` if the element is not found; `discard()` removes an element if present, but does nothing if not found.

22. Set Union and Intersection Problem: Given two sets `set_a = {1, 2, 3, 4}` and `set_b = {3, 4, 5, 6}`, find their union and intersection. *Solution:*

```

set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

union_set = set_a.union(set_b) # or set_a | set_b
intersection_set = set_a.intersection(set_b) # or set_a & set_b

print(f"Union: {union_set}")          # Output: {1, 2, 3, 4, 5, 6}
print(f"Intersection: {intersection_set}") # Output: {3, 4}

```

Explanation: `union()` (or `|`) combines all unique elements. `intersection()` (or `&`) finds common elements.

23. Set Difference and Symmetric Difference Problem: Using `set_a` and `set_b` from the previous question, find the elements in `set_a` but not in `set_b` (difference) and elements unique to each set (symmetric difference). *Solution:*

```

set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

difference_ab = set_a.difference(set_b) # or set_a - set_b
symmetric_difference = set_a.symmetric_difference(set_b) # or set_a ^ set_b

print(f"Difference (A - B): {difference_ab}") # Output: {1, 2}
print(f"Symmetric Difference: {symmetric_difference}") # Output: {1, 2, 5}

```

Explanation: `difference()` (or `-`) returns elements only in the first set.

`symmetric_difference()` (or `^`) returns elements in either set, but not in both.

24. Checking Subsets and Supersets Problem: Determine if `set_c = {1, 2}` is a subset of `set_a = {1, 2, 3, 4}`. Determine if `set_a` is a superset of `set_c`. *Solution:*

```

set_a = {1, 2, 3, 4}
set_c = {1, 2}

is_subset = set_c.issubset(set_a) # or set_c ≤ set_a
is_superset = set_a.issuperset(set_c) # or set_a ≥ set_c

print(f"{set_c} is a subset of {set_a}: {is_subset}") # Output: True
print(f"{set_a} is a superset of {set_c}: {is_superset}") # Output: True

```

Explanation: `issubset()` checks if all elements of one set are in another. `issuperset()` checks the inverse.

25. Removing Duplicates from a List using Set Problem: Write a function `unique_elements(input_list)` that takes a list and returns a new list with all duplicate elements removed using a set. *Solution:*

```

def unique_elements(input_list):
    return list(set(input_list))

print(unique_elements([1, 2, 2, 3, 1, 4])) # Output: [1, 2, 3, 4] (order is not guaranteed)

```

Explanation: Converting a list to a set automatically removes duplicates. Converting it back to a list gives a list of unique elements. *Pitfall:* This method does not preserve the original order of elements.

26. Frozen Sets *Problem:* Explain what a `frozenset` is and provide a simple example of when it might be useful. *Solution:*

```
my_frozenset = frozenset([1, 2, 3])

# Frozensets can be used as dictionary keys because they are immutable
dictionary_with_frozenset_key = {my_frozenset: "This is a frozenset key"}
print(dictionary_with_frozenset_key)

# Attempting to add to a frozenset will raise an error
try:
    my_frozenset.add(4)
except AttributeError as e:
    print(f"Error: {e}") # Output: Error: 'frozenset' object has no attril
```

Explanation: A `frozenset` is an immutable version of a set. Because it cannot be changed after creation, it is hashable and can be used as a dictionary key or as an element of another set. Regular sets are mutable and thus not hashable.

27. Set Comprehension *Problem:* Use set comprehension to create a set of squares of odd numbers from 1 to 10. *Solution:*

```
odd_squares = {x**2 for x in range(1, 11) if x % 2 != 0}
print(odd_squares) # Output: {1, 9, 25, 49, 81}
```

Explanation: Similar to list comprehensions, set comprehensions provide a concise way to create sets. The curly braces `{}` indicate a set, and the `if` condition filters for odd numbers.

28. Counting Unique Words *Problem:* Write a function `count_unique_words(text)` that takes a string of text and returns the number of unique words in it (case-insensitive). *Solution:*

```
import re

def count_unique_words(text):
    words = re.findall(r'\b\w+\b', text.lower()) # Extract words and convert to lowercase
    return len(set(words))

text = "Hello world, hello Python, world is great!"
print(f"Unique words: {count_unique_words(text)}") # Output: Unique words: 7
```

Explanation: We use `re.findall` with a regular expression to extract words, convert them to lowercase to treat “Hello” and “hello” as the same word, and then use a set to count unique occurrences.

29. Membership Testing Efficiency Problem: Demonstrate the efficiency of the `in` operator for membership testing within a large set compared to a list. *Solution:*

```
import time

large_list = list(range(1_000_000))
large_set = set(range(1_000_000))

# Check in list
start_time = time.time()
_ = 999_999 in large_list
end_time = time.time()
print(f"Time to check in list: {end_time - start_time:.6f} seconds")

# Check in set
start_time = time.time()
_ = 999_999 in large_set
end_time = time.time()
print(f"Time to check in set: {end_time - start_time:.6f} seconds")
```

Explanation: Membership testing (`in` operator) in sets is, on average, $O(1)$ (constant time) because sets use hash tables. In contrast, for lists, it's $O(n)$ (linear time) because Python might have to iterate through the entire list. For large collections, sets are significantly faster for membership checks.

30. Multi-Set Intersection Problem: Given three separate sets, identify the specific elements that are present in every one of them. *Solution:*

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
s3 = {4, 6, 7, 8}

common_elements = s1.intersection(s2, s3) # or s1 & s2 & s3
print(f"Common to all three: {common_elements}") # Output: {4}
```

Explanation: The `intersection()` method can take multiple sets as arguments, finding elements common to all of them. The `&` operator can also be chained for the same purpose.

IV. Utilizing Tuples

31. Tuple Creation and Access (Immutability) Problem: Create a tuple `coordinates = (10, 20)`. Access and print the first element. Try to modify an element and explain the outcome. *Solution:*

```
coordinates = (10, 20)
print(f"Original coordinates: {coordinates}")
print(f"First element: {coordinates[0]}") # Output: 10

try:
    coordinates[0] = 15
except TypeError as e:
    print(f"Error: {e}") # Output: Error: 'tuple' object does not support
```

Explanation: Tuples are immutable, meaning their elements cannot be changed after creation. This immutability is a key difference from lists and makes tuples suitable for data that should remain constant, providing data integrity.

32. Tuple Unpacking Problem: Given a tuple `person = ("Alice", 30, "New York")`, unpack its elements into three separate variables `name`, `age`, and `city`. *Solution:*

```

person_info = ("Alice", 30, "New York")
name, age, city = person_info
print(f"Name: {name}, Age: {age}, City: {city}")

```

Explanation: Tuple unpacking allows assigning elements of a tuple to multiple variables simultaneously. The number of variables on the left must match the number of elements in the tuple.

33. Tuples as Dictionary Keys *Problem:* Create a dictionary where keys are tuples representing coordinates (e.g., (x, y)) and values are names of locations. *Solution:*

```

location_map = {
    (0, 0): "Origin",
    (10, 20): "Park Entrance",
    (5, -5): "Fountain"
}

print(location_map[(10, 20)]) # Output: Park Entrance

```

Explanation: Tuples are hashable (due to their immutability), which means they can be used as keys in dictionaries. Lists, being mutable, cannot be used as dictionary keys.

34. Returning Multiple Values from a Function *Problem:* Write a function that returns a tuple of two values, and demonstrate how to unpack them. *Solution:*

```

def get_dimensions():
    return 100, 200 # Python automatically packs this into a tuple

width, height = get_dimensions()
print(f"Width: {width}, Height: {height}") # Output: Width: 100, Height: 200

```

Explanation: Functions can return multiple values by separating them with commas. Python automatically packs these values into a tuple, which can then be unpacked by the caller.

35. Tuple Concatenation and Repetition *Problem:* Concatenate two tuples `tup1 = (1, 2)` and `tup2 = (3, 4)`. Repeat `tup1` twice. *Solution:*

```
tup1 = (1, 2)
tup2 = (3, 4)

combined_tuple = tup1 + tup2
repeated_tuple = tup1 * 2

print(f"Combined: {combined_tuple}") # Output: (1, 2, 3, 4)
print(f"Repeated: {repeated_tuple}") # Output: (1, 2, 1, 2)
```

Explanation: The `+` operator creates a new tuple by concatenating two tuples. The `*` operator creates a new tuple by repeating the elements of the original tuple a specified number of times.

36. Checking Element Existence in Tuple Problem: Check if "apple" is in ("apple", "banana", "cherry"). *Solution:*

```
fruits_tuple = ("apple", "banana", "cherry")
print("apple" in fruits_tuple) # Output: True
print("grape" in fruits_tuple) # Output: False
```

Explanation: The `in` operator works for tuples just as it does for lists and sets, efficiently checking for membership.

37. Counting Elements in Tuple Problem: Count the occurrences of 5 in (1, 5, 2, 5, 3, 5). *Solution:*

```
my_tuple = (1, 5, 2, 5, 3, 5, 1)
count_of_5 = my_tuple.count(5)
print(f"Count of 5: {count_of_5}") # Output: 3
```

Explanation: The `count()` method is a convenient way to find the frequency of a specific element within a tuple or list.

38. Nested Tuples Problem: Create a nested tuple `matrix = ((1, 2), (3, 4))`. Access the element 4. *Solution:*


```
matrix_tuple = ((1, 2), (3, 4), (5, 6))
element = matrix_tuple[1][1] # Access second tuple, then its second element
print(element) # Output: 4
```

Explanation: Accessing elements in nested tuples works similarly to nested lists, using chained index brackets.

39. Single Element Tuple Syntax Problem: How do you create a tuple with a single element, e.g., (5,)? Explain why (5) is not a tuple. *Solution:*

```
single_element_tuple = (5,)
print(f"Type of (5,): {type(single_element_tuple)}") # Output: <class 'tuple'>

not_a_tuple = (5)
print(f"Type of (5): {type(not_a_tuple)}") # Output: <class 'int'>
```

Explanation: For a tuple with a single element, a trailing comma is essential. Without it, Python interprets (5) as an integer 5 enclosed in parentheses, not a tuple. The comma signals to Python that it is a tuple constructor.

40. Sorting a List of Tuples Problem: Given a list of tuples `grades = [('Alice', 90), ('Bob', 75), ('Charlie', 95)]`, sort it by the second element (grade) in descending order. *Solution:*

```
grades = [('Alice', 90), ('Bob', 75), ('Charlie', 95)]
grades.sort(key=lambda item: item[1], reverse=True)
print(grades) # Output: [('Charlie', 95), ('Alice', 90), ('Bob', 75)]
```

Explanation: We use the `sort()` method with a `key` (a lambda function that extracts the grade, which is the second element `item[1]`) and `reverse=True` to sort in descending order. This is a common pattern for sorting complex data structures.

V. Working with Dictionaries

41. Dictionary Creation and Access *Problem:* Create a dictionary `person = {"name": "John", "age": 30, "city": "New York"}`. Access and print the person's age. Add a new key-value pair for "occupation": "Engineer". *Solution:*

```
person = {"name": "John", "age": 30, "city": "New York"}
print(f"Age: {person['age']}") # Output: Age: 30
person["occupation"] = "Engineer"
print(person) # Output: {'name': 'John', 'age': 30, 'city': 'New York', 'occupation': 'Engineer'}
```

Explanation: Dictionary values are accessed using their keys in square brackets. New key-value pairs are added by assigning a value to a new key.

42. Iterating Through Dictionary Items *Problem:* Iterate through the `person` dictionary and print each key-value pair. *Solution:*

```
person = {"name": "John", "age": 30, "city": "New York"}
for key, value in person.items():
    print(f"{key.capitalize()}: {value}")
```

Explanation: The `.items()` method returns a view of key-value pairs, allowing for convenient iteration using a `for` loop.

43. Using `get()` with a Default Value *Problem:* Try to access a non-existent key "salary" from the `person` dictionary using `person['salary']` and then using `person.get('salary', 'N/A')`. Explain the difference. *Solution:*

```

person = {"name": "John", "age": 30, "city": "New York"}

# Using direct access (will raise KeyError)
try:
    print(person['salary'])
except KeyError as e:
    print(f"Error with direct access: {e}") # Output: Error with direct access: 'salary'

# Using .get() with a default value
salary = person.get('salary', 'N/A')
print(f"Salary using .get(): {salary}") # Output: Salary using .get(): N/A

```

Explanation: Direct access `person[key]` raises a `KeyError` if the key is not found. The `person.get(key, default_value)` method returns the value for the key if it exists, otherwise it returns the `default_value` (or `None` if no default is specified), preventing errors.

44. Dictionary Comprehension Problem: Use dictionary comprehension to create a dictionary where keys are numbers from 1 to 5 and values are their squares. *Solution:*

```

squares_dict = {x: x**2 for x in range(1, 6)}
print(squares_dict) # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

```

Explanation: Dictionary comprehensions provide a concise way to create dictionaries. The syntax is `{key_expression: value_expression for item in iterable}`.

45. Merging Dictionaries Problem: Merge two dictionaries `dict1 = {"a": 1, "b": 2}` and `dict2 = {"b": 3, "c": 4}`. If there are common keys, `dict2`'s values should override `dict1`'s. *Solution:*

```
dict1 = {"a": 1, "b": 2}
dict2 = {"b": 3, "c": 4}

# Method 1: Using update()
merged_dict_update = dict1.copy() # Create a copy to avoid modifying dict1
merged_dict_update.update(dict2)
print(f"Merged with update(): {merged_dict_update}") # Output: {'a': 1, 'b': 3, 'c': 4}

# Method 2: Using ** (Python 3.5+)
merged_dict_unpack = {**dict1, **dict2}
print(f"Merged with **: {merged_dict_unpack}") # Output: {'a': 1, 'b': 3, 'c': 4}
```

Explanation: The `update()` method merges a dictionary into another, overwriting values for common keys. The `**` operator (dictionary unpacking) creates a new dictionary from multiple dictionaries, with later dictionaries overriding earlier ones for common keys.

46. Counting Word Frequencies *Problem:* Write a function `word_frequency(sentence)` that takes a sentence and returns a dictionary where keys are words and values are their frequencies (case-insensitive). *Solution:*

```
import re

def word_frequency(sentence):
    words = re.findall(r'\b\w+\b', sentence.lower()) # Extract words and convert to lowercase
    freq_dict = {}
    for word in words:
        freq_dict[word] = freq_dict.get(word, 0) + 1
    return freq_dict

sentence = "This is a test sentence. This sentence is a test."
print(word_frequency(sentence)) # Output: {'this': 2, 'is': 2, 'a': 2, 'test': 2, 'sentence': 2}
```

Explanation: We extract words, convert them to lowercase, and then use a dictionary to store counts. `freq_dict.get(word, 0)` safely retrieves the current count or 0 if the word hasn't been seen yet.

47. Nested Dictionaries *Problem:* Create a nested dictionary representing a student's grades in different subjects. Access and print the student's grade in 'Math'. *Solution:*

```

student_grades = {
    "name": "Emily",
    "grades": {
        "Math": 95,
        "Science": 88,
        "History": 92
    }
}

math_grade = student_grades["grades"]["Math"]
print(f"Emily's Math grade: {math_grade}") # Output: Emily's Math grade: 95

```

Explanation: Nested dictionaries are accessed by chaining key lookups. This structure is common for representing hierarchical data.

48. Deleting a Key-Value Pair Problem: From the `person` dictionary, remove the “city” key-value pair. *Solution:*

```

person = {"name": "John", "age": 30, "city": "New York", "occupation": "Engineer"}
del person["city"]
print(person) # Output: {'name': 'John', 'age': 30, 'occupation': 'Engineer'}

```

Explanation: The `del` keyword is used to remove a key-value pair from a dictionary. Attempting to delete a non-existent key will raise a `KeyError`.

49. Dictionary `keys()`, `values()`, `items()` Views Problem: Given a dictionary `data = {"a": 1, "b": 2, "c": 3}`, print its keys, values, and items as lists. *Solution:*

```

data = {"a": 1, "b": 2, "c": 3}

print(f"Keys: {list(data.keys())}") # Output: ['a', 'b', 'c']
print(f"Values: {list(data.values())}") # Output: [1, 2, 3]
print(f"Items: {list(data.items())}") # Output: [('a', 1), ('b', 2), ('c', 3)]

```

Explanation: `keys()`, `values()`, and `items()` return dictionary view objects, which are dynamic. Converting them to `list()` allows you to see their current state as a list.

50. Inverting a Dictionary *Problem:* Write a function `invert_dict(d)` that takes a dictionary and returns a new dictionary where keys become values and values become keys. Assume all values are unique and hashable. *Solution:*

```
def invert_dict(d):  
    return {value: key for key, value in d.items()}  
  
my_dict = {"a": 1, "b": 2, "c": 3}  
inverted = invert_dict(my_dict)  
print(inverted) # Output: {1: 'a', 2: 'b', 3: 'c'}
```

Explanation: This uses dictionary comprehension to efficiently swap keys and values. The assumption that values are unique and hashable is crucial, as new keys must be unique and hashable.

References

1. [Python Documentation: Data Structures](#)
2. [Python Documentation: Defining Functions](#)
3. [Python Documentation: Sets](#)
4. [Python Documentation: Tuples and Sequences](#)
5. [Python Documentation: Dictionaries](#)